

CSCE 633 MACHINE LEARNING · MODULE 12 · LECTURE NOTES

How Evaluations Lie

The ways a number can be right and mean nothing.

MODULE OUTCOMES

- Identify the forms of leakage and audit for them.
- Explain multiple-comparisons inflation and correct for it.
- Explain benchmark overfitting and the adaptivity problem.
- Recognise Simpson's paradox and subgroup effects.
- Design an evaluation that is hard to fool.

1 Leakage

Form	How it gets in	Symptom
Target leakage	A feature whose value is determined after, or by, the outcome — a 'date closed' field, a treatment that is only given to confirmed cases.	One feature dominates the importance ranking and the score is near-perfect. The most visible form, and the easiest to catch if you look.
Train–test contamination	Preprocessing fitted on the full dataset — scaling, imputation, selection, PCA (Module 04 §2).	A small optimistic bias that is hard to see and easy to introduce; the result still looks plausible.
Temporal leakage	A random split on data with a time ordering, so the model trains on the future.	Excellent offline performance and immediate failure live — the characteristic signature.
Group leakage	The same entity contributes rows to both train and test (Module 01 §2).	Inflated and invisible unless you know the grouping key, which is sometimes not even in the dataset.
Duplicate rows	The identical record appears in both sets.	Very common in scraped, merged, or augmented data, and trivial to check for.
Proxy leakage	A feature that encodes the label indirectly — a patient ID range, a file path, an image border.	The hardest to find, because the feature is individually innocuous.

The classic example: a model trained to detect disease from chest images performed superbly and was found to be reading the *scanner type* — because the sickest patients were imaged on a portable machine, which left a distinctive marker. **The model was right about the correlation and useless for the purpose,** and it would have failed completely at any hospital with different equipment. Variants of this have been

found repeatedly across medical imaging.

THE LEAKAGE AUDIT, AS A PROCEDURE

For every feature, ask: would this value have been available, with this value, at the moment the prediction must be made? Write the answer down for each one, explicitly. The qualifier matters — a field that exists but is later updated is leakage.

Check the top of the feature importance ranking (Module 06 §4). **An implausibly dominant single feature is leakage far more often than it is a discovery**, and this check takes two minutes.

Check for duplicate and near-duplicate rows across the split — exact hashes first, then fuzzy matching if the data warrants it.

And be suspicious of good results. If the score is substantially better than the domain expert expected, the explanation is usually leakage rather than insight. **'We did better than expected' is not a conclusion; it is a prompt to audit** — and building that reflex is more valuable than memorising the taxonomy above.

2 Multiple comparisons

$$P(\text{at least one false positive}) = 1 - (1-\alpha)^m$$

For m independent tests at significance level α . **With $m = 20$ and $\alpha = 0.05$ that is 64% — try twenty things and the odds favour at least one spurious 'significant' result. With $m = 100$ it is 99.4%: you are essentially guaranteed a winner that is not real. A hyperparameter sweep is a multiple comparisons problem that nobody corrects for**, and so is comparing a dozen model families on the same validation set. Bonferroni and Benjamini–Hochberg corrections exist and are almost never applied in machine learning practice.

THE VALIDATION SET WEARS OUT

Every time you consult the validation set and then change something, you fit to it a little. Not through training — through *your own choices*, which are a learning algorithm with the validation score as its objective.

After a hundred such decisions the validation estimate is optimistic by an amount that nobody measures and that everybody incurs. This is the *adaptivity* problem, and it is the reason cross-validation scores reported after extensive tuning should be treated as upper bounds.

Which is exactly why the test set is spent once (Module 04 §2) — it is the only estimate that has not been contaminated by your attention, and it stops being so the moment you look at it and react.

And it is why benchmark leaderboards drift. The research community collectively overfits a public test set over years of published attempts, so replication on freshly collected data consistently finds smaller gains than the leaderboard progression suggested — which has been measured directly, on CIFAR and ImageNet, with the ranking largely preserved and the absolute numbers substantially lower.

3 Aggregates that reverse

	GROUP A (900 cases)	GROUP B (100 cases)	OVERALL
Model 1	81 / 90	192 / 910	273/1000 = 27%
Model 2	234 / 270	55 / 730	289/1000 = 29%

Per group:

Group A:	Model 1 = 90%	Model 2 = 87%	M1 better
Group B:	Model 1 = 21%	Model 2 = 8%	M1 better
OVERALL:	Model 1 = 27%	Model 2 = 29%	M2 "better"

Model 1 is better in EVERY group and worse overall, because the two were evaluated on different group MIXTURES.

Always report by SUBGROUP as well as overall. An aggregate hides composition, and composition changes between development and deployment -- exactly when the reversal bites.

REPORT BY SLICE, ALWAYS

An aggregate metric can conceal a model that fails completely for a subgroup while performing well on the majority that dominates the average. The aggregate is not wrong; it answers a question nobody asked.

So report the metric for every slice you can define — by customer segment, by data source, by time period, by device, by demographic where that is relevant and permitted. **And report the counts alongside**, so that a slice with eleven examples is discounted appropriately rather than panicked over.

The worst slice is frequently the number that actually matters, particularly when a person is on the receiving end of the prediction — which is Module 13's subject, approached from the measurement side rather than the ethical one.

And this is the same argument as CSCE 735 Module 07's insistence on distributions and CSCE 678 Module 13's on tail latency. The average describes nobody, and the worst case is somebody's experience. Three courses, three domains, one conclusion — which at this point in the program should be read as a general fact about reporting rather than a coincidence.

4 Designing an evaluation that is hard to fool

- **Write the evaluation protocol before seeing any results, and commit it with a timestamp. The highest-value item on this list**, because it makes post-hoc rationalisation visible to *you* — which is the audience that matters, since nobody sets out to deceive themselves.
- **Split by the unit you intend to generalise over**, and by time if the data has a time ordering (Module 01 §2). Decide this from 'what will be new at prediction time?' rather than from the library default.
- **Hold out a final set that you touch exactly once** — and genuinely once, which may require putting it somewhere inconvenient.

- **Report by slice, with counts** (§3), so that both the worst case and its sample size are visible.
- **Report the trivial baseline** (Module 01 §4) **and the uncertainty** (Module 04 §4) rather than a point estimate, so a reader can judge whether a difference is real.
- **And have someone else try to break it. A reader who did not build the pipeline finds leakage that you cannot**, because you know why each feature is there and they only know what it is. **This is CSCE 606's code review argument applied to evaluation**, and it works for the same reason.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Kaufman, Rosset & Perlich — Leakage in Data Mining (free)	The §1 taxonomy , with competition-winning examples that were leakage.
Kapoor & Narayanan — Leakage and the Reproducibility Crisis in ML-based Science (free)	How often §1 happens — surveyed across seventeen fields, with a checklist worth adopting.
Recht, Roelofs, Schmidt & Shamir — Do ImageNet Classifiers Generalize to ImageNet? (free)	The §2 benchmark-drift measurement . New test sets, collected the same way, and the accuracy drop.
Dwork et al. — The reusable holdout (free)	The adaptivity problem of §2, with a principled method for reusing a holdout set that bounds the damage.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Audit every feature in your project** against the availability question, and write the answers down.
- 2 **Deliberately introduce target leakage** and observe the score and the importance ranking.
- 3 Check your dataset for exact duplicate rows across the split.
- 4 **Simulate multiple comparisons**: generate pure noise, run 100 hypothesis tests, and count the 'significant' results.
- 5 **Tune a model on noise** with 200 hyperparameter configurations and report the best validation score obtained.
- 6 **Construct a Simpson's paradox** from your own data by choosing subgroups.
- 7 Report your project's metric by at least three slices, with counts.

- 8 **Find your worst slice** and state whether it would be acceptable in deployment.
- 9 Write and timestamp an evaluation protocol for a new problem before touching it.
- 10 **Have a classmate audit your pipeline for leakage** and report what they found.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name six forms of leakage and the symptom of each.
- 2 State the leakage audit procedure.
- 3 Why should a surprisingly good result be audited?
- 4 Compute the false positive probability for 20 and 100 tests.
- 5 Why does the validation set wear out, and what follows?
- 6 Why do benchmark leaderboards drift?
- 7 Construct a Simpson's paradox and explain the mechanism.
- 8 Why report by slice, and what connects this to CSCE 735 and 678?
- 9 Give six elements of a hard-to-fool evaluation.